# Environnement de développement intégré

## Première année BTS

Réalisé par P.LAHLOU Amine

Année scolaire 2025-2026

LAHLOU AMINE



# Chapitre 5 : Programmation événementielle

LAHLOU AMINE



# Séquence 1: Contrôles, Evènements

LAHLOU AMINE



## Programmation évènementielle

**Définition:**

La programmation événementielle en informatique repose sur les événements et diffère de la programmation séquentielle. Dans ce paradigme, le programme réagit principalement aux événements tels que les clics de souris ou les saisies clavier. Il peut être considéré comme une méthode architecturale où une boucle principale qui détecte et gère les événements, notamment dans le contexte des interfaces graphiques

LAHLOU AMINE

## Principe de la Programmation évènementielle

En plus de la programmation séquentielle, C# permet d'implémenter la programmation par événements, où des procédures réagissent à des événements déclenchés, souvent par l'utilisateur. Nous nous concentrons ici sur les applications à fenêtres multiples (Windows Applications Form), qui se composent de deux éléments essentiels :

1. la partie visuelle, comprenant les fenêtres, boutons et étiquettes visibles par l'utilisateur final,
2. et le code C#, situé en "arrière-plan," que le développeur doit créer.

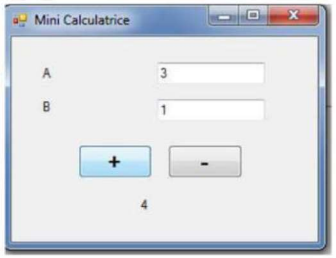


LAHLOU AMINE

## Principe de la Programmation évènementielle

**Exemple Mini-Calculatrice:**

1. L'utilisateur de l'application va interagir avec la fenêtre de la mini-calculatrice.
2. L'utilisateur saisi deux entiers A et B
3. Il clique sur un bouton d'addition ou de soustraction
4. Le système affiche le résultat.





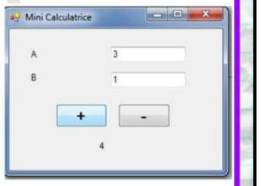

LAHLOU AMINE

## Principe de la Programmation évènementielle

**Exemple Mini-Calculatrice:**

Quand on clique sur le bouton de la somme, on voit apparaître le résultat.

—> **Le clique sur le bouton est un événement qui a causé l'affichage de la somme.**




Quand un événement a lieu, une procédure liée à cet événement est appelée. Le développeur de l'application doit développer le code C# relatif à ces procédures. Une procédure événementielle est donc une procédure classique mais qui s'exécute quand un événement particulier se produit.

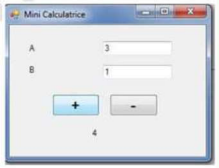


LAHLOU AMINE

## Principe de la Programmation évènementielle

**Exemple Mini-Calculatrice:**

Dans notre exemple, on doit par exemple développer la procédure au clique sur le bouton de l'addition. Cette procédure va :




1) Récupérer l'entier A saisi par l'utilisateur
2) Récupérer l'entier B saisi par l'utilisateur
3) Faire la somme
4) Afficher le résultat.



LAHLOU AMINE

## Principe de la Programmation évènementielle

En pratique, la procédure événementielle générée pour un objet suit la syntaxe suivante

```
Private void ButtonName_Click(object sender, EventArgs e) {

}
```

Exemple:

Nom de la procédure

Évènement

référence vers l'objet qui a déclenché l'événement.

```
private void btnAddition_Click(object sender, EventArgs e)
{
}
```

C'est un objet qui contient des informations supplémentaires sur l'événement.



## Rappel: Windows Form Application (Objets)

La partie visuelle du projet est composée principalement d'un ou de plusieurs formulaires (Forms). Un formulaire représente une fenêtre (**Window**). Dans ce formulaire, on va ajouter les **éléments** graphiques dont on a besoin.

| Nom de l'élément | Utilité |
|---|---|
| **Form** (feuille) | C'est le conteneur graphique des contrôles de l'application |
| **Label** (étiquette) | Afficher un texte statique : un libellé. |
| ***Text Box*** (zone de texte) | rentrer une valeur saisie, on peut aussi l'utiliser pour afficher du texte mais ce n'est pas recommandé. (utiliser plutôt le label pour l'affichage) |



## Rappel: Windows Form Application (Objets)

| | |
|---|---|
| **Button** (bouton de commande) | Lancer l'exécution d'une procédure événementielle (généralement suite au click sur ce bouton) |
| ListBox | Afficher une liste de valeurs. |
| ComboBox | Combiner l'utilité des contrôles ListBox et TextBox |
| PictureBox | Afficher une image dans un cadre. (Il peut être redimensionné en fonction de l'image en utilisant Autosize = True) |
| RadioButton (bouton radio) | Sélectionner **une seule** option parmi plusieurs. |
| Check Box (case à cocher) | choisir **une ou plusieurs** options parmi d'autes. |
| ***GroupBox*** | Regroupe un ensemble de contrôles graphique dans un seul cadre au sein du formulaire. |



## Rappel: Windows Form Application (Objets)

C# est un langage orienté objet. Les éléments graphiques (qu'on appelle des contrôles), sont des objets de classes:

**Par exemple:** un bouton nommé "bouton!" est une instance de la Classe "System.Windows.Forms.Button".

Un bouton a donc des propriétés (des caractéristiques, comme les attributs des objets) et des méthodes prédéfinies qu'on peut utiliser.

Pour chaque élément, on doit changer les propriétés des éléments. Il s'agit généralement de changer 2 propriétés principales: "nom" et "texte". Ceci se fait à partir de la fenêtre "propriétés".

**Le nom** : sert à désigner l'élément graphique dans le code et Le **texte** : c'est le texte à afficher dans l'interface.



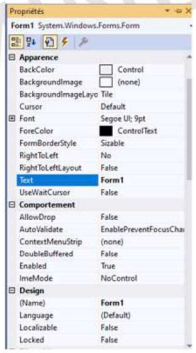
LAHLOU AMINE

## Rappel: Windows Form Application (Propriétés)

Chaque élément graphique a des propriétés communes avec les autres éléments, et d'autres qui lui sont spécifiques.

Voici quelques propriétés communes couramment utilisées:

- **Name** : nom interne de l'objet utilisé dans le code
- **Text** : Texte affiché à l'écran (sauf pour les champs de saisie)
- **BackColor** : couleur du fond
- **Enabled** : Contrôle activé ou non
- **font** : police de caractères
- **foreColor** : couleur de l'écriture
- **Visible** : Contrôle visible ou non
- **Width** : Largeur de l'objet
- **Height** : Hauteur de l'objet






## Windows Form Application (Contrôler les Propriétés)

Pour accéder à une propriété à partir du code C#, on utilise la syntaxe suivante:

**NomObjet.Propriété**

**Exemples:**

txtNom.Text="moi"; —> modifier le texte du « textBox » qui s'appelle « txtNom »

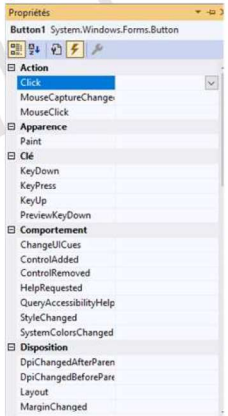
cmdQuitter.enabled =false; —> rendre le bouton qui s'appelle « cmdQuitter » désactivé.



## Windows Form Application (Events)

Dans C# Chaque objet possède une ou plusieurs procédures évènementielles possibles(Méthodes).

Pour créer les instructions à exécuter suite à l'appelle de la procédure évènementielle voulue, Il suffit de double cliquer sur l'évènement choisi.






## Windows Form Application (Contrôler les Évènements)

Pour accéder au corps d'une méthode à partir du code C#, on utilise la syntaxe suivante:

**NomObjet.Méthode ()**

**Exemples :**

frmPrincipale.hide(); —> cacher la fenêtre, elle n'est plus visible

frmPrincipale.show(); —> rendre la fenêtre visible



LAHLOU AMINE

## Exercice 1

Créer le code qui permet de changer le **texte** d'une application C# de type « Windows application Form » lors de son **chargement**.

```
private void Form1_Load(object sender, EventArgs e)
        {
            this.Text = "nouveau titre";
        }
```

LAHLOU AMINE



## Exercice 2

Concevoir une IHM « **Windows Application Form** » vide puis coder les évènements suivants:

L'évènement **Load** change le texte du formulaire par 'LOAD EVENT' et la couleur de la forme par le 'Rouge'

L'évènement **Click** change le texte du formulaire par 'CLICK EVENT' et la couleur de la forme par le 'Bleu'

L'évènement **DoubleClick** change le texte //// par 'DOUBLECLICK EVENT' et la couleur de la forme par le 'Jaune'

L'évènement **KeyPress** change le texte du formulaire par 'KEYPRESS EVENT' et la couleur de la forme par le 'Noir'

L'évènement **Move** change le texte du formulaire par 'MOVE EVENT' et la couleur de la forme par le 'Rose'

LAHLOU AMINE



## Exercice 2

```
private void Form1_Load(object sender, EventArgs e)
{
    this.Text = "LOAD EVENT";
    this.BackColor = Color.Red;
}

private void Form1_Click(object sender, EventArgs e)
{
    this.Text = "Click EVENT";
    this.BackColor = Color.Blue;
}

private void Form1_DoubleClick(object sender, EventArgs e)
{
    this.Text = "Double Click EVENT";
    this.BackColor = Color.Yellow;
}

private void Form1_KeyPress(object sender, KeyPressEventArgs e)
{
    this.Text = "KEYPRESS EVENT";
    this.BackColor = Color.Black;
}

private void Form1_Move(object sender, EventArgs e)
{
    this.Text = "Move EVENT";
    this.BackColor = Color.Pink;
}
```

LAHLOU AMINE